

# Lab 02 — Praktijklab: een image ontleden, een registry opzetten

*Doel: tags, lagen en digests hanteren, en dan een image publiceren in een private registry die je zelf laat draaien — en onderweg ontdekken waarom Podman je de namen voluit laat schrijven.*

**Vereisten** — Lab 01 afgewerkt (rootless Podman onder WSL, `systemd` actief). Poort `5000` moet vrij zijn (`ss -lntp | grep :5000` mag niets teruggeven).

**Geleverde bestanden** — `files/Dockerfile` (twee regels, uitgelegd in lab 04; hier dient het alleen als imagegenerator).

## Stap 1 — Een imagenaam lezen

```
podman pull nginx:alpine
podman pull alpine
```

**Observeer** `Resolved "nginx" as an alias`, dan `Trying to pull docker.io/library/nginx:alpine...`, de regels `Copying blob`, en voor `alpine` enkel een identificatie: de image is er al sinds lab 01.

```
podman image inspect --format '{{.RepoTags}}' nginx:alpine
podman image inspect --format '{{.Digest}}' nginx:alpine
```

**Observeer** enerzijds `[docker.io/library/nginx:alpine]` — de **volledige** naam, die je niet getypt hebt — anderzijds `sha256:1f25fedd50aec27413031afb...`.

*Uitleg.* `nginx:alpine` is een leesbare, verplaatsbare naam; de digest is de echte, permanente identiteit van de inhoud. Podman toont altijd de volledige naam: in zijn opslag bestaat er geen "korte naam", alleen in jouw commando.

Probeer nu een naam die niet in de aliaslijst staat:

```
grep -c '=' /etc/containers/registries.conf.d/000-shortnames.conf
grep -E '^s*"(\alpine|nginx|eclipse-temurin)"' /etc/containers/registries.conf.d/000-
shortnames.conf
grep unqualified-search-registries /etc/containers/registries.conf
```

**Observeer** dat `alpine` en `nginx` een alias hebben, `eclipse-temurin` niet, en dat de zoeklijst van Ubuntu alleen `docker.io` bevat — daarom werkt `podman pull eclipse-temurin:21-jre-alpine` bij jou toch, terwijl het op Fedora een vraag zou stellen.

### PODMAN

Een korte naam is een gemak in de terminal, geen bedrijfspraktijk. Schrijf in een Dockerfile of een script `docker.io/library/eclipse-temurin:21-jre-alpine`: het resultaat hangt dan niet meer af van de configuratie van de machine die het uitvoert.

## Stap 2 — Je images ophijsten

```
podman images
```

**Observeer** de kolommen `REPOSITORY / TAG / IMAGE ID / CREATED / SIZE`, en repositories die voluit geschreven zijn: `docker.io/library/nginx`, `docker.io/library/alpine`.

```
podman images --format 'table {{.Repository}}\t{{.Tag}}\t{{.Size}}'
podman images --format '{{.Repository}}:{{.Tag}}'
```

*Uitleg.* Maak er een gewoonte van `--format` te gebruiken: het maakt je commando's onafhankelijk van wijzigingen in de weergave, en bruikbaar in scripts.

## Stap 3 — De lagen, en waar het gewicht naartoe gaat

```
podman history nginx:alpine --format 'table {{.Size}}\t{{.CreatedBy}}'
```

**Observeer** één grote laag (50.7MB, de installatie van nginx), enkele kleine COPY-lagen van scripts, een laag van 8.7MB (Alpine) helemaal onderaan, en veel regels van 0B.

*Uitleg.* De regels van 0B zijn **metadata**-instructies: `ENV`, `CMD`, `EXPOSE`, `ENTRYPOINT`, `STOPSIGNAL`. Ze maken geen enkel bestand aan. Dit commando is je eerste reflex wanneer een image abnormaal zwaar is: de schuldige regel springt eruit.

Podman kan de lagen ook als een boom tonen, met hun bronimage:

```
podman image tree nginx:alpine
```

**Observeer** de eerste laag, gemarkeerd `Top Layer of: [docker.io/library/alpine:latest]`: nginx:alpine is **gebouwd op** de alpine-image die je al hebt — die laag van 8,7 MB wordt maar één keer opgeslagen.

```
podman system df
podman system df -v | head -n 12
```

**Observeer** in de uitgebreide modus de kolom `SHARED SIZE`: 8.698MB voor `nginx` en voor `alpine`, dezelfde laag in beide meegeteld.

## Stap 4 — `podman tag` kopieert niets

```
podman tag nginx:alpine mijn-nginx:v1
podman tag nginx:alpine mijn-nginx:preprod
podman images --format 'table {{.Repository}}\t{{.Tag}}\t{{.ID}}' | grep nginx
```

**Observeer** drie regels... met **dezelfde** `IMAGE ID` — en je twee nieuwe namen met het voorvoegsel `localhost/`.

```
podman rmi mijn-nginx:v1
```

**Observeer** de uitvoer: alleen `Untagged: localhost/mijn-nginx:v1`. Geen `Deleted: .`

```
podman rmi mijn-nginx:preprod
```

**Observeer** opnieuw alleen `Untagged: —` want `docker.io/library/nginx:alpine` verwijst nog altijd naar de image.

*Uitleg.* Een tag is een verwijzing. Zolang er een naam overblijft, blijft de data. Daarom is "ik heb `rmi` 's gedaan en geen plaats teruggekregen" een frequente en volkomen normale klacht. En dat `localhost/`: een image die je zonder registry benoemt, hoort bij geen enkele registry, en Podman zegt dat.

## Stap 5 — Twee versies van dezelfde image bouwen

Kopieer het geleverde bestand naar een werkmap:

```
mkdir -p ~/labo-docker/02 && cd ~/labo-docker/02
cp <pad-van-het-lab>/files/Dockerfile .
cat Dockerfile
```

```
podman build -t demo-lagen:1.0 .
podman history demo-lagen:1.0 --format 'table {{.ID}}\t{{.Size}}\t{{.CreatedBy}}'
```

**Observeer** de regels `STEP 1/2`, `STEP 2/2`, `COMMIT demo-lagen:1.0`, `Successfully tagged localhost/demo-lagen:1.0`, en dan drie lagen: je `RUN` ( `2.05kB` ), de `CMD` van de basisimage ( `0B` ), en het Alpine-bestandssysteem ( `8.7MB` ), gemarkeerd `<missing>` omdat die laag bij een andere image hoort.

Wijzig de versie en bouw opnieuw op **dezelfde tag**:

```
sed -i 's/version 1/version 2/' Dockerfile
podman build -t demo-lagen:1.0 .
podman run --rm demo-lagen:1.0 cat /version.txt
```

**Observeer** `version 2`, en een nieuwe `IMAGE ID` voor dezelfde tag.

```
podman images --filter dangling=true
```

**Observeer** een regel `<none> <none>` met de **oude** `IMAGE ID`: dat is de *dangling* image, die haar naam verloor. Ze neemt nog altijd 8,7 MB in (gedeeld, in dit geval).

*Uitleg.* De tag `demo-lagen:1.0` is **verplaatst** naar een nieuwe image: precies het scenario van vraag 2. Niets waarschuwt de gebruiker. Verwijder het residu via zijn identificatie:

```
podman rmi $(podman images --filter dangling=true -q)
```

## Stap 6 — Een private registry opzetten

Een registry is niets anders dan een container:

```
podman run -d -p 5000:5000 --name lab-registry registry:2
podman ps --filter name=lab-registry
curl -s http://localhost:5000/v2/_catalog
```

**Observeer** `0.0.0.0:5000->5000/tcp` in de kolom `PORTS`, en dan `{"repositories":[]}`: de registry is leeg en werkt.

#### → WINDOWS / WSL

Die poort 5000 is gepubliceerd in de WSL-VM, maar Windows ziet ze ook: open `http://localhost:5000/v2/_catalog` in je Windows-browser. WSL 2 stuurt poorten waarop in Linux geluisterd wordt automatisch door naar `localhost` aan Windows-zijde (*localhost forwarding*). Zo zul je in de volgende labs de Angular-frontend kunnen testen vanuit Edge of Chrome.

Publiceer er je image in:

```
podman tag demo-lagen:1.0 localhost:5000/basis/demo:1.0
podman push localhost:5000/basis/demo:1.0
```

**Observeer** de mislukking:

```
Error: ... pinging container registry localhost:5000: Get "https://localhost:5000/v2/":
http: server gave HTTP response to HTTPS client
```

*Uitleg.* Je registry spreekt HTTP; Podman eist standaard HTTPS, **zelfs voor localhost** — waar Docker stilzwijgend een uitzondering maakt. Voor een testregistry zeg je het expliciet:

```
podman push --tls-verify=false localhost:5000/basis/demo:1.0
curl -s http://localhost:5000/v2/_catalog
curl -s http://localhost:5000/v2/basis/demo/tags/list
```

**Observeer** de regels `Copying blob`, `Writing manifest to image destination`, en dan `{"repositories":["basis/demo"]}` en `{"name":"basis/demo","tags":["1.0"]}`.

#### → BEVEILIGING

Het alternatief is de registry declareren in `~/.config/containers/registries.conf` (`[[registry]]`, `location = "localhost:5000"`, `insecure = true`). Handig op een ontwikkelwerkpost, en gevaarlijk overall elders: een "insecure" registry is er een waarvan noch de identiteit noch de versleuteling geverifieerd wordt, dus een waarin een aanvaller op het netwerk een image kan vervangen. In een bedrijf heeft een registry een certificaat, punt.

Je hebt zonet in drie commando's nagedaan wat de CI van je bedrijf doet. Ga de differentiële overdracht na:

```
podman tag demo-lagen:1.0 localhost:5000/basis/demo:1.1
podman push --tls-verify=false localhost:5000/basis/demo:1.1
```

**Observeer** dat dezelfde blobs vermeld worden maar dat de overdracht onmiddellijk is: de registry heeft ze al, alleen het manifest wordt geschreven.

---

## Stap 7 — Pullen op digest

Haal de digest op zoals de registry hem kent:

```
curl -sI -H "Accept: application/vnd.oci.image.manifest.v1+json" \
  http://localhost:5000/v2/basis/demo/manifests/1.0 | grep -i docker-content-digest
```

**Observeer** een regel `Docker-Content-Digest: sha256:239accdd...`. Kopieer die waarde.

```
podman rmi localhost:5000/basis/demo:1.0
podman pull --tls-verify=false localhost:5000/basis/demo@sha256:<plak_hier>
podman images --format 'table {{.Repository}}\t{{.Tag}}\t{{.ID}}\t{{.Digest}}' | grep demo
```

**Observeer** dat de image gepulld is en dat `podman image inspect --format '{{.Digest}}' demo-lagen:1.0` precies de waarde geeft die je zonet plakte.

*Uitleg.* Dit is de vorm die ernstige uitrollen gebruiken: ze is **onvervalsbaar**. Zelfs als iemand `basis/demo:1.0` opnieuw publiceert met een andere inhoud, blijft jouw digest verwijzen naar de image die je getest hebt.

## Stap 8 — Een image vervoeren zonder netwerk

```
podman save -o /tmp/demo.tar demo-lagen:1.0
ls -lh /tmp/demo.tar
tar -tf /tmp/demo.tar | head -n 4
```

**Observeer** een archief van `8.4M` met `<sha256>.tar`-bestanden (de lagen), een `.json` (de configuratie) en een `manifest.json`: het historische formaat `docker-archive`.

```
podman save --format oci-archive -o /tmp/demo-oci.tar demo-lagen:1.0
tar -tf /tmp/demo-oci.tar | head -n 4
```

**Observeer** deze keer `blobs/sha256/...` en `index.json`: de **OCI**-indeling, die alle tools (Docker, Podman, skopeo, Kubernetes) lezen.

Vergelijk met `export`, dat op een **container** werkt:

```
podman run -d --name tmpx nginx:alpine
podman export tmpx | podman import - nginx-plat:v1
podman image inspect --format '{{json .Config.Cmd}}' nginx-plat:v1
podman run --rm nginx-plat:v1
```

**Observeer** `null`, en dan de fout `crun: cannot find `` in $PATH`: de geïmporteerde image **weet niet meer wat ze moet starten**.

*Uitleg.* Onthoud de regel: `save` / `load` voor een image, `export` / `import` nooit voor een uitrol.

```
podman rmi demo-lagen:1.0 localhost:5000/basis/demo:1.1
podman load -i /tmp/demo.tar
```

**Observeer** `Loaded image: localhost/demo-lagen:1.0`: de tag is met het archief teruggekomen.

---

## Opruimen

```
podman rm -f -t 0 tmpx lab-registry
podman rmi nginx-plat:v1 demo-lagen:1.0 registry:2
podman images --format '{{.Repository}}:{{.Tag}}' | grep -E 'demo|plat|registry'
rm -f /tmp/demo.tar /tmp/demo-oci.tar
```

De image die in stap 7 op digest gepulld is, kan overblijven:

```
podman images --format 'table {{.ID}}\t{{.Repository}}' | grep localhost:5000
podman rmi <ID>
```

**Observeer** dat alleen `docker.io/library/alpine` en `docker.io/library/nginx:alpine` overblijven, bewaard voor het vervolg.

---

## Wat je nu moet kunnen beweren

- Een tag is een verplaatsbare verwijzing; de digest identificeert een inhoud. Podman bewaart en toont volledige namen, en zet `localhost/` voor de jouwe.
- `podman history` en `podman image tree` onthullen waar het gewicht van een image naartoe gaat en wat ze deelt.
- `podman tag` dupliceert niets; `podman rmi` verwijdert eerst een naam.
- Een `push` draagt alleen de lagen over die in de registry ontbreken.
- Een registry is een gewone HTTP-dienst, met één commando op te zetten — maar Podman eist TLS, tenzij `--tls-verify=false` expliciet gezegd is.
- `export` / `import` vernietigt de configuratie van de image; `save` / `load` bewaart ze, in `docker-archive` - of `oci-archive` -formaat.